ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Estudo e Implementação de Uma Estufa Inteligente de Pequena Dimensão

Eduardo Miguel Moreira Junqueira e Pedro Miguel Ferreira Pinto

Mestrado em Engenharia Electrotécnica e de Computadores
Área de Especialização em Automação e Sistemas

Orientador: André Vaz Da Silva Fidalgo

Universidade Técnica do Porto
Instituto Superior de Engenharia do Porto
Departamento de Engenharia Eletrotécnica

Novembro, 2026

Estudo e Implementação de Uma Estufa Inteligente de Pequena Dimensão

Dissertação submetida por **Eduardo Miguel Moreira Junqueira e Pedro Miguel Ferreira Pinto** (estudante nº 1251561 e 1200919) para obtenção do Grau de Mestre em Engenharia Electrotécnica e de Computadores, Área de Especialização em Automação e Sistemas.

Orientador:

André Vaz Da Silva Fidalgo, anf@isep.ipp.pt

Co-Orientador:

António José Matos De Meireles, aem@isep.ipp.pt

Júri:

Presidente:

Maria Isabel de Castro Lopes Martins Pinto Ferreira, DEE/ISEP

Vogais:

Nome do vogal, DDD/PPPP

Nome do vogal, DEE/ISEP

ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Rua Dr. António Bernardino de Almeida 431, 4249–015 Porto

Novembro, 2026

ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Declaração de Integridade

Eu, Eduardo Miguel Moreira Junqueira e Pedro Miguel Ferreira Pinto, estudante n.º 1251561 e 1200919, declaro ter conduzido este trabalho académico com integridade. Não plagiei ou apliquei qualquer forma de uso indevido de informações ou falsificação de resultados ao longo do processo que levou à sua elaboração.

Declaro que o trabalho apresentado neste documento é original e de minha autoria, não tendo sido utilizado anteriormente para nenhum outro fim. Adicionalmente declaro que:

☒ não utilizei quaisquer ferramentas de inteligência artificial generativa na elaboração desta dissertação;

☐ utilizei ferramentas de inteligência artificial generativa na elaboração desta dissertação, conforme descrito no Anexo "Declaração de Uso de Inteligência Artificial Generativa", do qual assumo plena responsabilidade.

Declaro ainda que tenho pleno conhecimento do Código de Conduta Ética da UTP.

ISEP, Porto, 11 de Novembro de 2026

Resumo

No âmbito do *software*, propõe-se o desenvolvimento de uma aplicação móvel para o sistema *Agriculture Internet of Things* (AgrIoT), que permitirá ao utilizador monitorizar e controlar remotamente a estufa através de uma *User Interface* (UI) simples e acessível. A aplicação comunicará com um servidor através de uma *Application Programming Interface* (API) sobre *HyperText Transfer Protocol* (HTTP), com troca de dados em formato *JavaScript Object Notation* (JSON), de modo a integrar os valores medidos pelos sensores e os comandos de atuação do sistema *Internet of Things* (IoT). Neste documento são apresentados o projeto da arquitetura de *software*, os módulos que a compõem, os procedimentos e vetores de teste previstos e a respetiva análise de custos.

Palavras-Chave: *Internet of Things*, Aplicação *Android*, Monitorização e Controlo.

Abstract

Regarding the software, the development of a mobile application for the *Agriculture Internet of Things* (AgrIoT) system is proposed, allowing the user to remotely monitor and control the greenhouse through a simple and accessible *User Interface* (UI). The application will communicate with a server through an *Application Programming Interface* (API) over *HyperText Transfer Protocol* (HTTP), exchanging data in *JavaScript Object Notation* (JSON) format, in order to integrate the values measured by the sensors and the actuation commands of the IoT system. This document presents the software architecture design, the modules that compose it, the planned test procedures and vectors, and the corresponding cost analysis.

Keywords: Internet of Things, Android Application, Monitoring and Control.

Índice

Lista de Figuras **ix**

Lista de Tabelas **xi**

Lista de Acrónimos **xiii**

1 Software **1**

1.1 Linguagens e Tecnologias 1
1.1.1 Aplicação *Android* 1
1.1.2 Servidor: *Python* e *Flask* 2
1.1.3 Comunicação: HTTP, API e JSON 2
1.1.4 Base de Dados: *SQLite* 3
1.1.5 Resumo 4
1.2 Conceção da Interface (*Mockups*) 5
1.2.1 Visão Geral dos Ecrãs 5
1.2.2 Identidade Visual e Componentes 6
1.2.3 Ecrãs do Utilizador 6
1.2.4 Perfil de Administrador 7
1.3 Arquitetura de Software 8
1.3.1 Visão Geral 8
1.3.2 Fluxo de Dados 8
1.3.3 Modelo de Dados 8
1.3.4 Fluxo de Navegação 8
1.4 Módulos do Sistema 8
1.4.1 Módulos da Aplicação 8
1.4.2 Módulos do Servidor 8
1.4.3 Módulo de Controlo 8
1.4.4 Organização dos Módulos 8
1.5 Procedimentos e Vetores de Teste 8
1.5.1 Testes da Aplicação 8
1.5.2 Testes da API e da Base de Dados 8
1.5.3 Testes de Integração 8
1.6 Lista de Componentes e Análise de Custos 8

Referências **9**

Lista de Figuras

1.1 *Android developers use Kotlin* [1]. 2
1.2 Logótipo *framework Flask* [6]. 2
1.3 Redirecionamentos em HTTP [7] 3
1.4 Arquitetura de transporte de dados *JavaScript Object Notation* (JSON) [8] 3
1.5 Logótipo *SQL database engine* [9] 4
1.6 Conjunto dos *mockups* da aplicação *Agriculture Internet of Things* (AgrIoT). 5

Lista de Tabelas

1.1 Resumo das linguagens e tecnologias utilizadas. 4
1.2 Ecrãs do administrador e operação *HyperText Transfer Protocol* (HTTP) associada. 7

Lista de Acrónimos

AgrIoT	*Agriculture Internet of Things*
API	*Application Programming Interface*
HTTP	*HyperText Transfer Protocol*
IDE	*Integrated Development Environment*
IoT	*Internet of Things*
JSON	*JavaScript Object Notation*
PID	Proporcional-Integral-Derivativo
REST	*Representational State Transfer*
SDK	*Software Development Kit*
SGBD	Sistema de Gestão de Bases de Dados
SQL	*Structured Query Language*
TCP	*Transmission Control Protocol*
UI	*User Interface*
URL	*Uniform Resource Locator*

Capítulo 1

Software

Este capítulo descreve a componente de *software* do sistema *Internet of Things* (IoT) desenvolvido para a monitorização e o controlo da estufa inteligente nomeado deAgrIoT. O *software* constitui a camada responsável por recolher os dados medidos pelos sensores, armazená-los, processá-los e disponibilizá-los ao utilizador, bem como por transmitir os comandos de atuação ao *hardware* e controlar os mesmos.

1.1 Linguagens e Tecnologias

Esta secção define as linguagens, *frameworks* e protocolos utilizados no desenvolvimento do *software*, de forma a enquadrar as decisões de arquitetura apresentadas na Secção 1.3.

1.1.1 Aplicação *Android*

O *Android* é um sistema operativo *open-source*, baseado no cerne *Linux*, concebido para dispositivos móveis, como telemóveis,*tablets*, entre outros, e constitui a principal alternativa ao sistema *iOS* da *Apple*. O respetivo *Software Development Kit* (SDK) disponibiliza as bibliotecas, as ferramentas de compilação e os emuladores necessários ao desenvolvimento de aplicações para esta plataforma [1].

A aplicação será desenvolvida em *Kotlin*, uma linguagem de programação criada pela *Google I/O* em 2019 com suporte a *Android*, na figura seguinte 1.1: [2].



Figura 1.1: *Android developers use Kotlin* [1].

Como ambiente de desenvolvimento, será utilizado o *Android Studio*, na versão *Rabbit 2*, que é o *Integrated Development Environment* (IDE) oficial para esta plataforma. Este ambiente integra o editor de código, o editor visual de *layouts*, o emulador de dispositivos e as ferramentas de depuração, permitindo concentrar num único *software* as etapas de desenvolvimento, teste e compilação da aplicação [3].

Antes da implementação da aplicação, serão desenvolvidos *mockups* dos ecrãs, isto é, representações visuais da interface que permitem definir o aspeto e a organização dos elementos antes de escrever código. Para esse efeito será utilizado a ferramenta *Figma* de *design* de interfaces, que permite conceber, prototipar e partilhar a *User Interface* (UI) de aplicações *web* e móveis [4].

Entre as principais funcionalidades do *Figma* destacam-se a criação de ecrãs (*frames*) com as dimensões de dispositivos reais, a definição de componentes reutilizáveis, como botões e campos de texto, e a prototipagem interativa, que permite simular a navegação entre ecrãs sem escrever código [4].

1.1.2 Servidor: *Python* e *Flask*

O *Python* é uma linguagem de programação de alto nível, com uma vasta coleção de bibliotecas [5]. O *Flask* é uma *framework* para o desenvolvimento de aplicações *web* e móvel em *Python*, caracterizada pela sua simplicidade e por permitir definir rotas que respondem a pedidos HTTP, representado em baixo na figura 1.2 [6].



Figura 1.2: Logótipo *framework Flask* [6].

Neste projeto, o *Flask* é utilizado para implementar a *Application Programming Interface* (API) que faz a ligação entre a aplicação *android*, a base de dados e o *hardware*.

1.1.3 Comunicação: HTTP, API e JSON

O HTTP é o protocolo de comunicação, que é controlado na camada de transporte e portanto utiliza o padrão *Transmission Control Protocol* (TCP) do tipo pedido-resposta, utilizado na *web*, representado em baixo na figura 1.3 [7].

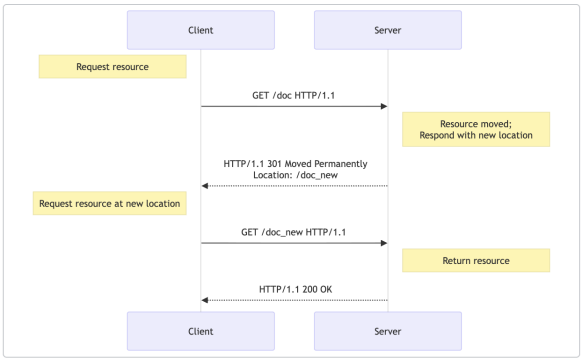


Figura 1.3: Redirecionamentos em HTTP [7]

.

Uma API define o conjunto de pedidos que um cliente (utilizador) pode efetuar a um servidor, e o fluxo arquitetural *Representational State Transfer* (REST) organiza esses pedidos em torno de recursos identificados por *Uniform Resource Locator* (URL). Os *webhooks* são mecanismos de comunicação em *real-time*, onde um aplicativo envia solicitações HTTP do tipo *POST* para uma URL predefinida. Os dados são trocados em formato JSON, um formato textual, independente da linguagem de programação [8]

Na figura que se segue explicita melhor o mecanismo de *webhook* 1.4 [8]

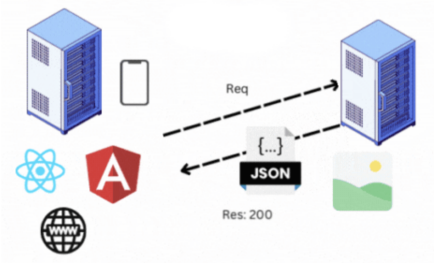


Figura 1.4: Arquitetura de transporte de dados JSON [8]

1.1.4 Base de Dados: *SQLite*

O *SQLite* é um Sistema de Gestão de Bases de Dados (SGBD) relacional, de código aberto, que implementa a linguagem *Structured Query Language* (SQL) e se destaca por funcionar embebido na própria aplicação, sem necessidade de um servidor de base de dados dedicado nem de configuração prévia [9]. Toda a base de dados é guardada num único ficheiro, o que facilita a sua utilização, cópia e manutenção, tornando-o adequado a aplicações móveis e a sistemas embebidos com recursos limitados, a figura demonstra o logótipo desta ferramenta: 1.5 [9] .



Figura 1.5: Logótipo *SQL database engine* [9]

.

Neste projeto, o *SQLite* será utilizado para guardar o histórico dos valores medidos pelos sensores e o estado dos atuadores em *real-time*.

1.1.5 Resumo

Em síntese, o *software* nomeado por AgrIoT é composto por três camadas principais: uma aplicação *android*, desenvolvida em *Kotlin* no *Android Studio* e precedida de *mockups* elaborados no *Figma*, que funciona como interface com o utilizador (Subsecção 1.1.1); um servidor desenvolvido em *Python* com a *framework Flask*, que expõe os serviços necessários através de uma API sobre HTTP (Subsecções 1.1.2 e 1.1.3); e uma base de dados *SQLite*, responsável pelo registo dos valores medidos e do estado dos atuadores (Subsecção 1.1.4). A troca de informação entre as diferentes camadas é realizada através de mensagens em formato JSON, e a forma como estes elementos se articulam é detalhada na Secção 1.3.

A Tabela 1.1 apresenta o resumo das tecnologias selecionadas, a camada do sistema em que cada uma intervém e a respetiva função.

Tabela 1.1: Resumo das linguagens e tecnologias utilizadas.

Tecnologia	**Camada**	**Função**
Figma	Conceção	*Mockups* e prototipagem dos ecrãs
Kotlin + XML	Aplicação	Lógica e interface da aplicação *android*
Android Studio	Aplicação	Desenvolvimento, teste e compilação da aplicação
Python + Flask	Servidor	API e processamento dos dados
HTTP + JSON	Comunicação	Transporte e formato dos dados
SQLite	Dados	Armazenamento persistente

Todo o código-fonte do *software* desenvolvido (aplicação *android*, servidor *Flask* e *scripts* da base de dados), bem como os *mockups* e a documentação associada, encontra-se guardado num repositório público do *GitHub*, o que permite acompanhar a evolução do projeto, manter o histórico de alterações e garantir a reprodutibilidade do trabalho [10].

1.2 Conceção da Interface (*Mockups*)

Antes de iniciar a implementação da aplicação *android*, foram desenvolvidos no *Figma* os *mockups* de todos os ecrãs do AgrIoT. Esta fase permitiu definir a organização da informação, a navegação e a identidade visual da aplicação, de forma a validar a solução antes de escrever código e a reduzir alterações durante o desenvolvimento. Os ecrãs foram desenhados com as dimensões de um telemóvel (412×915 píxeis), em orientação vertical.

1.2.1 Visão Geral dos Ecrãs

A Figura 1.6 apresenta o conjunto dos nove ecrãs desenhados. Os cinco primeiros correspondem ao utilizador comum: iniciar sessão, página inicial, sensores, atuadores e controlador Proporcional-Integral-Derivativo (PID). Os quatro restantes pertencem ao perfil de administrador e cobrem a gestão de utilizadores: lista, adição, edição e eliminação.

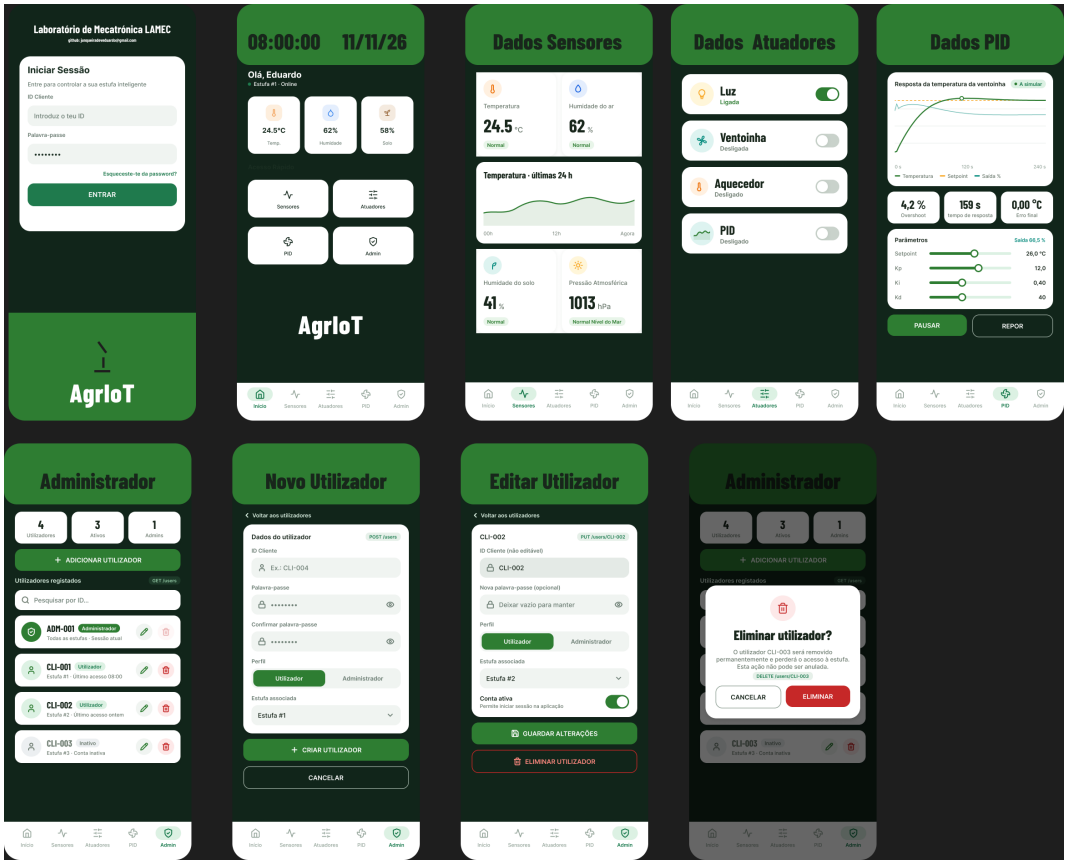


Figura 1.6: Conjunto dos *mockups* da aplicação AgrIoT.

1.2.2 Identidade Visual e Componentes

A interface segue uma linguagem visual única em todos os ecrãs, associada ao tema agrícola do projeto:

- **Cores:** fundo verde escuro, verde mais claro para a barra superior e para as ações principais, e cartões brancos que destacam a informação. O vermelho é usado apenas em ações destrutivas, como eliminar um utilizador.
- **Tipografia:** *Barlow Condensed* nos títulos e nos valores numéricos, para os destacar, e *Inter* nos restantes textos, pela legibilidade em ecrãs pequenos.
- **Barra superior:** faixa verde com cantos inferiores arredondados, que mostra o título do ecrã (ou a hora e a data, na página inicial).
- **Barra de navegação inferior:** fixa e comum aos cinco ecrãs principais (Início, Sensores, Atuadores, PID e Admin), com o separador ativo destacado.
- **Cartões, *chips* e interruptores:** os cartões agrupam a informação, os *chips* indicam estados (por exemplo, "Normal" ou "A simular") e os interruptores ligam e desligam os atuadores.

A reutilização destes componentes garante coerência entre ecrãs e facilita a implementação posterior, já que cada elemento passa a corresponder a um componente *XML* reutilizável.

1.2.3 Ecrãs do Utilizador

Iniciar Sessão. Ecrã de entrada, com o nome do laboratório, os campos "ID Cliente" e "Palavra-passe", a ligação "Esqueceste-te da password?" e o botão "ENTRAR". A parte inferior mostra o logótipo AgrIoT.

Página Inicial. Mostra a hora e a data, a saudação ao utilizador e o estado da estufa ("Online"). Apresenta ainda um resumo com a temperatura, a humidade do ar e a humidade do solo, e um bloco de acesso rápido às áreas Sensores, Atuadores, PID e Admin.

Sensores. Quatro cartões mostram os valores atuais de temperatura, humidade do ar, humidade do solo e pressão atmosférica, cada um com um *chip* de estado. Por baixo, um gráfico mostra a evolução da temperatura nas últimas 24 horas.

Atuadores. Lista os atuadores (luz, ventoinha, aquecedor e PID) com o estado "Ligada" ou "Desligada" e um interruptor para os comandar.

PID. Mostra o gráfico da resposta da temperatura da ventoinha, com o *setpoint* e a saída em percentagem. Apresenta ainda as métricas de desempenho (*overshoot*, tempo de assentamento e erro final) e deslizadores para ajustar o *setpoint* e os ganhos K_p, K_i e K_d. Os botões "PAUSAR" e "REPOR" controlam a simulação.

1.2.4 Perfil de Administrador

O separador *Admin* dá acesso à gestão de utilizadores, que segue as quatro operações de uma API REST (consultar, criar, atualizar e eliminar). A Tabela 1.2 relaciona cada ecrã com a operação HTTP correspondente.

Tabela 1.2: Ecrãs do administrador e operação HTTP associada.

Ecrã	**Operação**	**Função**
Administrador (lista)	*GET*	Consultar os utilizadores existentes, com pesquisa por ID
Novo Utilizador	*POST*	Criar utilizador (ID, palavra-passe, perfil e estufa)
Editar Utilizador	*PUT*	Alterar palavra-passe, perfil, estufa e estado da conta
Eliminar Utilizador	*DELETE*	Remover utilizador, com confirmação prévia

Lista. Apresenta o total de utilizadores, os ativos e os administradores, o botão "ADICIONAR UTILIZADOR" e uma pesquisa por ID. Cada utilizador surge num cartão com ID, perfil, estufa associada e estado, e com botões de editar e eliminar. A conta do administrador em sessão não pode ser eliminada, para evitar que o sistema fique sem administrador.

Novo e Editar Utilizador. Os dois formulários são semelhantes, o que facilita a implementação. Na edição, o ID aparece bloqueado, porque identifica o utilizador, e a nova palavra-passe é opcional. O perfil escolhe-se num seletor (Utilizador ou Administrador) e a conta pode ser ativada ou desativada com um interruptor.

Eliminar. Por ser uma ação irreversível, o pedido é confirmado numa janela sobre a lista, que escurece o fundo e usa o vermelho para destacar o botão de eliminação.

1.3 Arquitetura de Software

1.3.1 Visão Geral

1.3.2 Fluxo de Dados

1.3.3 Modelo de Dados

1.3.4 Fluxo de Navegação

1.4 Módulos do Sistema

1.4.1 Módulos da Aplicação

1.4.2 Módulos do Servidor

1.4.3 Módulo de Controlo

1.4.4 Organização dos Módulos

1.5 Procedimentos e Vetores de Teste

1.5.1 Testes da Aplicação

1.5.2 Testes da API e da Base de Dados

1.5.3 Testes de Integração

1.6 Lista de Componentes e Análise de Custos

Referências

[1] Android. *Kotlin and Android | Android Developers*. Nov. de 2026. URL: https://developer.android.com/kotlin (Citado nas páginas 1, 2).

[2] Jetbrains. *Kotlin for Android | Kotlin Documentation*. Nov. de 2026. URL: https://kotlinlang.org/docs/android-overview.html (Citado na página 1).

[3] Android. *Download Android Studio & App Tools - Android Developers*. Nov. de 2026. URL: https://developer.android.com/studio (Citado na página 2).

[4] Eduardo Junqueira. *The Collaborative Interface Design Tool LAMEC - AgrIoT – Figma*. Nov. de 2026. URL: https://www.figma.com/design/G2Ch3fnOpFUzGZ1Hv6wcUf/ (Citado na página 2).

[5] Python Software Foundation Legal Statements. *Welcome to Python.org*. Nov. de 2026. URL: https://www.python.org/ (Citado na página 2).

[6] Werkzeug WSGI toolkit. *Welcome to Flask — Flask Documentation*. Nov. de 2026. URL: https://flask.palletsprojects.com/en/stable/ (Citado na página 2).

[7] Mozilla. *Uma visão geral do HTTP - HTTP | MDN*. Nov. de 2026. URL: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview (Citado nas páginas 2, 3).

[8] Medium - Klaus Dellano. *As 6 Arquiteturas de API mais procuradas | by Klaus Dellano | HavanLabs | Medium*. Nov. de 2026. URL: https://medium.com/havanlabs/as-6-arquiteturas-de-api-mais-procuradas-25128fdbcd72 (Citado na página 3).

[9] SQLite Consortium members. *SQLite Home Page*. Nov. de 2026. URL: https://www.sqlite.org/ (Citado nas páginas 3, 4).

[10] GitHub - Eduardo Junqueira. *JunqueiraDevEduardo/AgrIoT: Aplicação Mobile LAMEC - Estufa Inteligente*. Nov. de 2026. URL: https://github.com/JunqueiraDevEduardo/AgrIoT (Citado na página 4).